

Drone Security Analyst Agent

Autonomous frame understanding, contextual reasoning, and alerting over a simulated drone surveillance feed

1. Overview and Approach

The system acts as an autonomous security analyst for an aerial patrol feed. It does not consume a live drone stream. Instead it samples frames from pre recorded CCTV footage and pairs each frame with synthetic telemetry (time of day, patrol waypoint, GPS, altitude) to simulate one. Every layer below blob storage is source agnostic, so swapping the simulated feed for a real stream later touches only the ingestion edge.

The problem was broken into four concerns, each independently runnable and verifiable before moving on: a data plane that produces frames and telemetry, a perception layer that turns an image into structured facts, a reasoning layer that decides whether a frame is routine or worth an alert, and an operator surface that makes all of it visible. The build followed an ordered sequence so that a live API existed early and every later stage integrated against a running system rather than isolated files.

The guiding design decision was to keep alert firing deterministic and the language model responsible only for description and narrative. A rule engine decides whether an alert fires, which makes behaviour reproducible and testable, while the model supplies the human readable situational summary. The model cannot suppress a rule that has fired.

2. Framework and Technology Stack

The stack favours mature, OpenAI compatible, container friendly components so the whole system runs from a single compose file.

Layer	Technology	Role
Agent framework	LangGraph 0.2	Stateful graph of reasoning nodes with conditional routing
Backend API	FastAPI + Uvicorn	Async REST surface and agent invocation
Database	PostgreSQL 15 + pgvector	Frames, events, alerts, and 1536 dim vector search
Blob storage	MinIO (S3 API, boto3)	Frame images, served to the browser by URL
Vision model	Groq Llama 4 Scout 17B	Frame to structured JSON description
Reasoning model	Groq Llama 3.3 70B	Situational narrative and noteworthiness
Chat reasoning	OpenAI gpt 4o mini	Q and A reasoning loop with a vision tool
Embeddings	OpenAI text embedding 3 small	Semantic search, local hashing fallback
Frame sampling	OpenCV (headless)	Sample frames from clips at an interval
Frontend	React 18 + Tailwind	Live operator dashboard
ORM / DB access	SQLAlchemy 2 async + asyncpg	Non blocking database calls

Testing	pytest + httpx	Unit, agent, and live API tests
---------	----------------	---------------------------------

Why these choices

- **LangGraph** models the per frame cycle as an explicit graph. Conditional edges express the normal, log, and alert routes cleanly, and the typed state object keeps each node small and independently testable.
- **pgvector inside PostgreSQL** avoids running a separate vector database. The same store holds structured rows and embeddings, so history retrieval and semantic search are one SQL dialect.
- **MinIO** gives an S3 compatible object store locally. Frame URLs are stored on each record, so the browser renders images directly without proxying through the API.
- **Groq for vision and reasoning** was chosen for low latency on the Llama 4 and Llama 3.3 models through an OpenAI compatible API, which let a single client abstraction serve both Groq and OpenAI with only a base URL and model name change.
- **OpenAI gpt 4o mini for the chatbot** keeps the interactive Q and A reasoning loop cheap while still supporting tool calling, which the loop uses to invoke the vision model on demand.
- **Deterministic rule engine** is deliberately separate from the model so security behaviour is reproducible run to run and can be unit tested without an API call.

3. System Design and Reasoning Flow

A single POST to the ingest endpoint runs one full reasoning cycle. The graph is wired with conditional edges so that a routine frame is simply persisted, a noteworthy frame is logged, and a rule violating frame is logged and alerted before persistence.

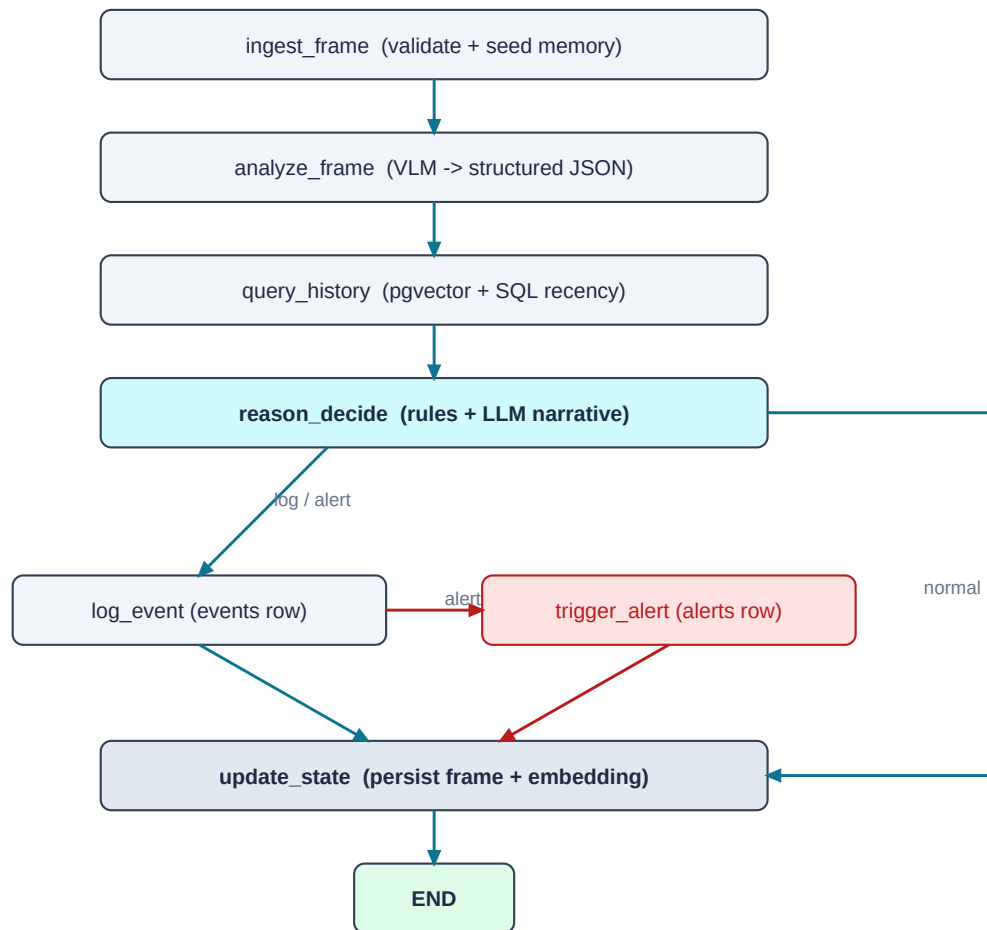


Figure 1. LangGraph agent. The route from *reason_decide* is normal, log, or alert.

Node responsibilities

- **ingest_frame** validates the frame URL and seeds working state from a process level session store that holds the rolling window of recent frames and the current session's events and alerts.
- **analyze_frame** fetches the image bytes from blob storage, base64 encodes them, and asks the vision model for a fixed schema: object type, location, action, clothing, colour, confidence, and a one sentence description.
- **query_history** embeds the description and runs two retrievals in parallel, a vector similarity search for similar past frames and a SQL filter for recent frames at the same waypoint, which feeds loitering and repeat detection.

- **reason_decide** runs the rule engine for the firing decision and the reasoning model for the narrative, then emits the route. If the model call fails it falls back to a deterministic summary so the pipeline never breaks.
- **log_event** and **trigger_alert** write the events and alerts rows, each carrying the triggering frame URL for the dashboard.
- **update_state** persists the frame with its embedding and telemetry, advances the session memory, and returns the API result.

Memory model

Graph state is per invocation, but the agent needs memory across frames. Two tiers handle this. A process singleton session store carries the rolling window of recent frame summaries and the current session's events and alerts, which drives cross frame rules such as loitering and repeat vehicle. PostgreSQL with pgvector holds durable history that query_history reads back on every frame.

Alert rules

Business hours default to 08:00 to 18:00. Severity is attached per rule.

Rule	Severity	Condition
after_hours_person	critical	Person detected outside business hours
after_hours_vehicle	high	Vehicle detected outside business hours
loitering	high	Same object and location across 5 or more consecutive frames
repeat_vehicle	warning	Same vehicle descriptor seen 3 or more times this session
restricted_zone	critical	Any subject inside a configured restricted location

4. Validation and Functionality Testing

Validation runs at three levels: unit and agent tests, a live API test suite, and a scenario harness that drives real video sourced frames through the running system and checks operator facing behaviour.

Automated test suites

- **Vision extraction test** asserts the analyze step returns the full schema with valid types and a usable embedding vector.
- **Agent pipeline test** runs a full cycle end to end and checks that frames, events, and alerts are written with a valid frame URL and embedding.
- **API test** exercises every REST endpoint against the live backend and asserts that frame URLs are present in search results and resolve to real images.

All nine automated tests pass against the running stack.

Dynamic input and emergency response scenarios

Twenty five frames were sampled from the source clips with deterministic seeded telemetry and driven through the live agent. Because the synthetic time of day is spread across a full 24 hour cycle, a single run produces both routine daytime frames and after hours frames, which is what exercises the emergency response path. The run produced 22 alert routes, 2 normal, and 1 log, firing after hours vehicle 10 times, after hours person 8 times, and repeat vehicle 8 times across 30 total frames in the database.

The scenario harness then checked nine cases. Result: 7 pass, 2 informational, 0 fail. The two informational cases are documented and expected behaviours of the dataset rather than defects.

Case	Scenario	Outcome
QA-01	Vehicle frame persisted with URL	Pass, 20 vehicle frames, all with URLs
QA-02	Repeat or after hours vehicle alert	Pass, 18 vehicle alerts
QA-03	After hours pedestrian alert	Pass, 8 alerts, e.g. person at loading_dock 04:48
QA-04	Loitering across 5 frames	Info, rule verified directly, see note
QA-05	Frame search returns images	Pass, query returns rows with URLs
QA-06	Search image is fetchable	Pass, HTTP 200, image/jpeg from MinIO
QA-07	Chat answer grounded in frames	Pass, answer cites a specific frame
QA-08	Session summary generated	Pass, summary over 30 frames and 26 alerts
QA-09	Normal daytime activity	Info, daytime alerts are repeat_vehicle by design

On QA-04, the loitering rule does not fire from the ingest run because the synthetic telemetry cycles the patrol waypoint on every frame, so no five consecutive same location frames occur. The rule itself was verified by direct evaluation with a constructed five frame window, which produced a loitering alert as expected. On QA-09, several daytime frames do raise alerts, but those are repeat_vehicle, which is intentionally not time gated since a vehicle making repeated passes is notable at any hour.

5. Results, with Examples

Frame descriptions, structured by the vision model

Each frame is reduced to a fixed schema. Representative outputs from the live run, exactly as stored:

Object	Location, time	Description
person	main_gate, 00:00	A woman and child walk through a mall corridor past storefronts and an orange construction barrier.
person	parking_lot, 04:00	A woman in a white shirt and dark pants walks through a shopping mall corridor.
car	perimeter_north, 14:24	A dark coloured car is driving in the center lane of a busy multi lane road.
car	perimeter_east, 15:21	A white car is driving on a busy road with multiple lanes.

Agent recommendations and narratives

The reasoning model produces a situational assessment per logged frame. Real examples from the run:

Critical, after_hours_person: "A person is walking in the corridor of a shopping mall, which is a routine activity, but the fact that a person was detected at the loading dock after hours triggers the after hours person rule."

Info, person_detected: "A person, wearing a white shirt and dark pants, is walking near storefronts in a mall corridor, which appears to be a routine activity given the location and time."

Session summary, generated over the full run

Headline: Multiple after hours vehicle sightings and repeat vehicle alerts indicate potential security concerns at the perimeter and warehouse areas.

Summary: The session recorded 26 alerts, with 10 after hours vehicle alerts and 8 repeat vehicle alerts. The most significant alerts were after hours vehicle sightings at various locations, including warehouse_a, parking_lot, and perimeter_east. A pattern of repeated after hours presence was observed, with multiple vehicles detected at different times.

Grounded question answering

The chatbot retrieves relevant frames, reasons over their metadata, and may call the vision model on a specific frame when the text index is not enough. Example exchange from the live system:

Q: Was a vehicle seen after hours?

A: Yes, a dark coloured car was seen parked at the main gate at 22:30, which is after hours. (one frame reference returned)

Demo video

The accompanying demo video walks through the running dashboard with these results live: the telemetry feed populating as frames arrive, the alert banner showing the triggering frame for an after hours detection, semantic frame search rendering thumbnails from stored URLs, the session summary, and the grounded chatbot. The video files are submitted alongside this report. Refer to them for the dashboard walkthrough and the per goal narration.

6. Challenges and Solutions

Several issues surfaced during integration and QA. Each is recorded here with the fix, since they shaped the final design.

Telemetry feed only showed alerted frames

The live telemetry panel polled the events endpoint, but events are written only for the log and alert routes. A normal frame with no alert produced no event row, so uploads with no alert never appeared. The fix added a frames listing endpoint that returns every ingested frame, and the feed now reads that. The event log panel keeps reading events, so the two panels have distinct, correct roles.

Single shot question answering missed visual detail

The first chatbot was a single retrieval and one model call over text metadata only. Questions needing visual detail not in the index could not be answered. The fix turned it into a short reasoning loop that can call the vision model on a specific frame when the metadata is insufficient, then fold the visual findings into the final answer.

Provider split for cost and latency

Vision and reasoning run on Groq for speed, while the interactive chatbot reasoning runs on a cheaper OpenAI mini model and embeddings run on OpenAI. The client abstraction passes an explicit provider per call so the split holds regardless of global provider toggles, and embeddings fall back to a local hashing embedder when no key is present.

Ingest runner crash on alert counting

The end to end runner aggregated fired rules with a counter, but fired rules are dictionaries, which are not hashable, so the run crashed on the first alerting frame. The fix counts by rule name. This was found by QA and is exactly why the runner is validated against a complete system rather than in isolation.

Dual MinIO endpoints

The backend container reaches MinIO by its service name, but the browser needs a host reachable URL. Storing a public endpoint URL on each record, separate from the internal endpoint used for byte access, lets the API and the browser both resolve frames correctly.

Loitering not observable from cycling telemetry

Because the synthetic telemetry cycles the waypoint every frame, the five consecutive same location condition is never met by an ingest run. Rather than weaken the rule, this was documented and the rule was verified directly, with a stationary telemetry mode noted as the way to demonstrate it end to end.

7. What Could Be Improved With More Time

The current system simulates a feed by sampling recorded clips. With more time the following directions would raise both realism and quality.

Live streaming and real time ingestion

- Replace the file sampler with a live capture path using OpenCV VideoCapture on an RTSP or device stream, so frames are pulled from an actual camera or drone link rather than from stored files.
- Add a streaming ingestion mode that pushes frames as they arrive, with backpressure and a sampling throttle so the vision model is not overwhelmed at full frame rate.
- Stream results to the dashboard over websockets for a true live telecast, replacing the current polling with server pushed updates and a live video tile alongside the telemetry feed.
- Real platform telemetry from the drone autopilot, replacing the synthetic time and GPS with genuine values, which also makes the after hours and zone rules act on real conditions.

Perception quality

- Fine tune or adapt the vision model on aerial and CCTV imagery if a suitably labelled dataset is available, since the general model can be uncertain on overhead views and small distant subjects. Even light domain adaptation on drone footage would improve object typing and action recognition.
- Noise removal and pre processing before the model sees a frame: denoising, deblurring, low light enhancement, and stabilisation, which directly affect description quality on real outdoor and night footage.
- Temporal smoothing and tracking across frames so the same subject is associated over time, which would make loitering and repeat detection far more robust than the current per frame descriptor matching.
- De duplication of near identical consecutive frames to save model calls and avoid repeated alerts for one continuous event.

Reasoning and operations

- A learned anomaly score to complement the deterministic rules, so subtle unusual patterns that no single rule captures can still be surfaced for review.
- Alert lifecycle management with acknowledgement, resolution, and de duplication so an operator is not flooded by repeated firings of the same situation.
- Confidence aware routing that asks for a second look from the vision model when the first description is low confidence, rather than acting on an uncertain read.
- Evaluation harness with labelled ground truth to measure precision and recall of the alerts, which the current scenario tests approximate but do not quantify.

Summary

The delivered system meets its core goal: it understands each frame, reasons over rolling and historical context, raises reproducible alerts with the triggering image, and presents everything through a live dashboard and a grounded question answering interface. The architecture keeps the language model bounded to perception and narrative while a deterministic engine owns the security decisions, which is

what makes the behaviour testable and the validation results above meaningful.